

Analyse Technique : TodoForm.vue

Comprendre le mécanisme de capture et d'émission

Le Template : Interaction Directe

```
v-model="newTask"  
placeholder="...">  
@click="submitTask">  
Ajouter
```

Points Clés :

-  v-model : Liaison bidirectionnelle. Ce qui est tapé va direct dans **newTask**.
-  @click : Écouteur d'événement qui déclenche la logique JS.

Le Script : État & Communication

```
export default {  
  emits: ['add-task'],  
  data() {  
    return {  
      newTask: ''  
    }  
  },  
}
```

Option emits

Déclaration de sécurité. On informe Vue que ce composant va envoyer un signal add-task au parent.

Option data

C'est l'entrepôt local. newTask stocke temporairement la saisie avant l'envoi.

Logique : La méthode submitTask

```
submitTask() {  
  if (this.newTask.trim() === '')  
    return  
  this.$emit('add-task', this.newTask.trim())  
  this.newTask = '';  
}
```

Le flux technique :

Trim : Nettoie les espaces inutiles.

\$emit : Le "haut-parleur". Envoie la donnée vers le composant parent.

Reset : Vide le champ pour la prochaine tâche.

Style Scoped : Isolation

```
<style scoped>
.todo-form {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}
```

Pourquoi "Scoped" ?

Vue ajoute un attribut unique (ex: data-v-123) aux éléments HTML de ce fichier.

Résultat : Le CSS de ce formulaire ne "bavera" jamais sur les autres boutons ou inputs de l'application.

TodoItem.vue

Le Template : Affichage et Événements

Le template gère ici la visualisation d'une tâche existante et capture les actions de l'utilisateur.

```
<template>
  <div class="todo-item" :class="{ done: task.done }">
    <input type = "checkbox"
      :checked = "task.done"
      @change = "$emit('toggle-task', task.id)"/>
    <span>{{ task.text }}</span>
    <button @click = "$emit('delete-task', task.id)">🗑️</button>
  </div>
</template>
```

Points Clés :

- 🔄 :class="{done: task.done}" : Liaison de classe dynamique. La classe CSSdone est appliquée uniquement si task.done est vrai.
- ✓ :checked="task.done" : Liaison unidirectionnelle. La case est cochée ou non selon l'état de la tâche reçue en prop".
- 🖱️ @change et @click : Écouteurs d'événements qui agissent comme des déclencheurs immédiats

Le Script : Réception et Signalement

```
<script>
export default {
  name: 'TodoItem',
  emits: ['toggle-task', 'delete-task'],
  props: {
    task: Object
  },
}
</script>
```

props: { task: Object } :

- C'est le canal d'entrée.
- Le parent "donne" l'objet tâche au composant. todoItem ne possède pas cette donnée, il l'emprunte pour l'afficher.

emits: ['toggle -task','delete -task']:

C'est la déclaration de sécurité.
On informe Vue que ce composant va envoyer deux signaux au parent.

La Logique : l'émission(\$emit)

Ici, pas de fonction intermédiaire complexe dans le script, tout se passe directement dans le template pour plus de rapidité:

```
@change = "$emit('toggle-task', task.id)"/>
<span>{{ task.text }}</span>
<button @click = "$emit('delete-task', task.id)">🗑️</button>
```

Le flux technique :

\$emit('toggle-task', task.id) :

- Le "Haut-parleur" : Envoie le signal de changement d'état.
- Le message : Il transmet l'identifiant (task.id) pour que le parent sache quelle tâche modifier.

\$emit('delete-task', task.id) :

Même principe, mais pour la suppression. Le composant dit au parent : "Supprime l'élément avec cet ID".

Style Scoped : Isolation

```
<style scoped>
.todo-item {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px;
  border-radius: 8px;
  margin-bottom: 8px;
  background-color: #f8f9fa;
}

.todo-item.done span {
  text-decoration: line-through;
  color: #aaa;
}
```

Pourquoi "Scoped" ?

Vue ajoute un attribut unique (ex: data-v-456) pour que le style du bouton poubelle, par exemple, ne s'applique qu'ici.

Style conditionnel : La règle `todo-item.done span` permet de barrer le texte (`line-through`) uniquement quand la tâche est marquée comme finie.

TodoList.vue

Composants

```
<script>
import TodoForm from './TodoForm.vue'
import TodoItem from './TodoItem.vue'
export default {
  name: 'TodoList',
  components: { TodoForm, TodoItem },
  data() {
    return {
      tasks: []
    }
  }
},
```

Components :

Déclare les composants enfants utilisable dans le template

Data : tasks []

Tableau qui stocke l'input de l'utilisateur,

mounted() : Chargement Initial

```
mounted() {  
  const saved = localStorage.getItem('tasks')  
  if (saved) {  
    this.tasks = JSON.parse(saved)  
  }  
},
```

S'exécute une seule fois au chargement de la page avec mounted()

localStorage.getItem()

Lit la valeur texte stockée dans le navigateur sous la clé "tasks"

If (saved)

.protège contre le cas où localStorage est vide, sans ce test, JSON.parse(null) planterait

JSON.parse(saved)

Reconvertit le texte stocké en vrai tableau JavaScript utilisable par Vue

mounted() : Chargement Initial

```
watch: {  
  tasks: {  
    deep: true,  
    handler(newTasks) {  
      localStorage.setItem('tasks', JSON.stringify(newTasks))  
    }  
  }  
},
```

Deep: true

Surveille les modifications internes au tableau (ex: cocher une tâche)

If (saved)

.Fonction déclenchée à chaque changement détecté

JSON.stringify()

Convertit le tableau en texte car localStorage ne stocke que du texte brut

Les méthodes : Logique CRUD

```
methods: {  
  addTask(texte){  
    this.tasks.push({ id:Date.now(), text: texte, done:false })  
  },  
  
  toggleTask(id){  
    const task = this.tasks.find(t => t.id === id)  
    if (task)  
      task.done = !task.done  
  },  
  
  deleteTask(id){  
    this.tasks = this.tasks.filter(t => t.id !==id)  
  }  
},  
}  
/script>
```

addTask(texte)

Crée un objet tâche avec un id unique (Date.now()) et l'ajoute au tableau tasks

toggleTask(id)

.Trouve la tâche par son id et inverse la valeur de **done** (true ->false ou l'inverse)

deleteTask(id)

Recrée le tableau sans la tâche avec la fonction **filter()**

Affichage dynamique

```
<todo-form @add-task = "addTask" />
<p class = "empty" v-if="tasks.length === 0">Aucune tâche pour l'instant !</p>

<transition-group name="fade" tag="div">
  <todo-item
    v-for="x in tasks"
    :key="x.id"
    :task="x"
    @toggle-task="toggleTask"
    @delete-task="deleteTask"
  />
</transition-group>
```

v-if

Affiche un message vide si
tasks.length === 0

v-for + :key

.Génère un TodoItem par
tâche

Transition-group

Anime l'entrée et la sortie des tâches
avec l'effet 'fade',

main.js et app.vue

Fichier : App.vue — Template & Script

```
<template>
  <div class="app-container">
    <h1>Ma Todo List</h1>
    <TodoList />
  </div>
</template>

<script>
import TodoList from './components/TodoList.vue'

export default {
  name: 'App',
  components: { TodoList }
}
</script>
```

<h1>

Affiche le titre 'Ma Todo List'.

<TodoList />

Appelle le composant TodoList depuis components/.

name: 'App'

Donne un nom au composant (utile pour le débogage Vue DevTools).

components: { TodoList }

Déclare TodoList pour l'utiliser dans le template

Fichier : App.vue — Section <style>

```
<style>
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background-color: #f0f2f5;
  font-family: Arial, sans-serif;
}

.app-container {
  max-width: 500px;
  margin: 40px auto;
  padding: 0 16px;
}

h1 {
  text-align: center;
  margin-bottom: 24px;
  color: #2c3e50;
}
</style>
```

*** { box-sizing, margin, padding }**

Réinitialise les marges et paddings de tous les éléments HTML.

body { background, font-family }

Fond gris clair #f0f2f5 et police Arial pour la page

.app-container { max-width }

Limite la largeur à 500px et centre le contenu.

h1 { text-align, color }

Centre le titre et lui donne une couleur bleu foncé #2c3e50.

Fichier : main.js

```
import { createApp } from 'vue'
import App from './App.vue'

const app = createApp(App)
app.mount('#app')
```

Points Clés :

createApp

Crée l'instance de l'application Vue.

import App

Importe App.vue, le composant racine de l'interface.

app.mount('#app')

Vue prend le contrôle du <div id='app'> dans index.html.